JALAAKAR
Command reference

Every command in the repository, one line each.

---

102 commands · **36** runnable scripts · **101** flags across **26** of them · **26** API endpoints

Generated from **COMMANDS.md** on 10 August 2026

Predicting when water runs out, 30 days early, from data India already publishes.

Run everything from the repository root. `PY` below means `.venv/bin/python` after `make setup`, or plain `python3` if you activated the venv yourself.

```
cd ~/Desktop/Jalakaar/jalaakar
PY=.venv/bin/python
```

1. Everyday

COMMAND	WHAT IT DOES
`make setup`	Creates `.venv`, installs both requirements files, builds `data/jalaakar.db` from `data/bootstrap/`. Run once after cloning.
`make run`	Serves the API and the whole website on http://localhost:8000.
`PORT=9000 make run`	Same, on a different port.
`make test`	90 API checks plus the frontend audit. The one to run before committing.
`make audit`	Frontend only — dead links, unstyled classes, stale cache stamps, unwired buttons.
`make demo-user`	Creates 5 demo accounts, password `jalaakar-demo`.
`make users`	Who registered and when, phone numbers masked.
`make delete-user PHONE=9123456780`	Removes one account, keeping its alert history.
`make delete-user PHONE=9123456780 DRY=1`	Shows what that would remove, without doing it.
`make whatsapp-check`	Will a send really deliver, or only render? Contacts nothing.
`make reset`	Wipes signups, alerts and sessions from `data/app.db`. Keeps the pipeline data.
`make clean`	Removes `.venv` and the built database.
`make`	Prints the target list.

No `make`? The three underlying commands:

```
python3 -m venv .venv && .venv/bin/pip install -r requirements.txt -r requirements-api.txt
.venv/bin/python tools/bootstrap.py
.venv/bin/uvicorn api.main:app --port 8000
```

2. Accounts and approvals

COMMAND	WHAT IT DOES
`$PY tools/verify_user.py --list`	Lists every account with its role and whether it may send. `NO` in the `ok` column is a pending approval.

COMMAND	WHAT IT DOES
`$PY tools/verify_user.py --phone 9800000001`	**Approves** a government account so it can send alerts. Use the real pending number from `--list`; `9800000001` is the demo official.
`$PY tools/verify_user.py --phone 9800000001 --revoke`	Withdraws that right and kills their live sessions immediately.
`$PY tools/list_users.py`	Who registered and when, newest first, phone numbers masked.
`$PY tools/list_users.py --role farmer`	Filter by role — `farmer`, `society-manager`, `society-resident`, `government`.
`$PY tools/list_users.py --place Baglan`	Filter by place, case-insensitive substring.
`$PY tools/list_users.py --since 2026-08-10`	Only accounts registered on or after that date.
`$PY tools/list_users.py --full`	Unmask the phone numbers. Real people's numbers — think before you do.
`$PY tools/list_users.py --csv > users.csv`	Same list as CSV for a spreadsheet.
`$PY tools/set_password.py --phone 9123456780`	Sets a password. Needed for accounts that predate passwords, and the only "I forgot it" path — there is no reset flow.
`$PY tools/delete_user.py --phone 9123456780 --dry-run`	Shows exactly what deleting one person would remove. Look first.
`$PY tools/delete_user.py --phone 9123456780`	Deletes that one account, after asking to confirm.
`$PY tools/delete_user.py --auto --dry-run`	Finds every account auto-created by a test send.
`$PY tools/reset_app_db.py --users --dry-run`	Shows what a full user wipe would remove.
`$PY tools/seed_demo.py`	Same as `make demo-user`.

Only government accounts need approving. Signup sets `verified=0` for them and `1` for everyone else, so a society manager can send the moment they register. An unverified official can still sign in — they just see "awaiting department verification" instead of the send controls.

3. Alerts and WhatsApp

COMMAND	WHAT IT DOES
`$PY tools/check_twilio.py`	Says whether a send will actually deliver or only render. Contacts nothing. Exit 0 = will deliver, 1 = render only.
`$PY tools/send_test_alert.py --phone 9123456780 --place Baglan --lang mr`	Walks the whole chain — place, score, message, send — and prints which step failed.
`$PY tools/send_test_alert.py ... --dry-run`	Renders the message without contacting Twilio.
`$PY tools/send_test_alert.py ... --force`	Sends even when the band is SAFE, which normally sends nothing.
`$PY tools/send_test_alert.py ... --role society-manager --on 2023-05-15`	Different template group, and a specific scoring date.

Real delivery needs credentials in the environment before `make run`:

```
export TWILIO_ACCOUNT_SID=AC...
export TWILIO_AUTH_TOKEN=...
export TWILIO_WHATSAPP_FROM='whatsapp:+14155238886'
```

Without them, alerts are **rendered and logged, never reported as delivered**. The recipient must send the sandbox join phrase from their own phone first, and the session expires after 72 hours of inactivity.

Inbound "Jal Mitra" messages need a public URL:

```
ngrok http 8000     # then point the Twilio sandbox webhook at /api/whatsapp/webhook
```

`POST /api/whatsapp/simulate` runs the identical flow offline if ngrok is a hassle on the day.

4. Data pipeline — in run order

You do not need any of these to run the site; `tools/bootstrap.py` builds enough. These rebuild from source.

COMMAND	WHAT IT DOES
`sqlite3 data/jalaakar.db < ingest/00_schema.sql`	Creates the schema — the frozen contract every other script reads.
`$PY ingest/01_diagnose.py`	Read-only look at the raw figshare download before loading anything.
`$PY ingest/01_figshare.py`	Downloads the peer-reviewed IISc groundwater dataset.
`$PY ingest/02_wells.py`	Loads `wells` and `gw_observations` — 940 wells, 68,994 real readings.
`$PY ingest/03b_nasapower.py`	Fetches NASA POWER daily weather per well into `weather_daily`. Add `--only-missing` to resume.
`$PY ingest/03_openmeteo.py --dry-run`	The Open-Meteo alternative. Check the request plan before spending the quota.
`$PY ingest/04_reservoirs.py --all`	Rebuilds `reservoirs` and `reservoir_daily` from `ingest/reservoir_seeds.csv`.
`$PY ingest/04_reservoirs.py --live`	Pulls today's published storage instead of the seeded anchors.
`$PY ingest/05_interpolate.py`	Builds `gw_daily`, the daily reconstruction. **Never train on this.**
`$PY ingest/05_interpolate.py --taluka Dindori --validate --plot --no-load`	Measures reconstruction error on held-out readings and plots one taluka, writing nothing.
`$PY ingest/06b_features_causal.py`	Builds `features_causal` — the only table safe to train on.
`$PY ingest/06_features.py`	The **leaky** older feature table. Kept for the baseline that proves the leak; do not train on it.
`$PY ingest/07_stress.py`	Computes `urban_stress`, the rule-based reservoir score.
`$PY ingest/07_stress.py --explain MUM_ALL --date 2026-06-29`	Shows the arithmetic behind one urban score.

COMMAND	WHAT IT DOES
`$PY ingest/07_stress.py --calibrate`	Prints the score against what BMC actually did in 2026.
`$PY ingest/db.py`	Smoke-tests the config and prints a row count per table. The quickest "is the database sane" check.

5. Model — in run order

Needs `features_causal`, so run `ingest/06b_features_causal.py` first.

COMMAND	WHAT IT DOES
`$PY ml/01_baseline.py`	The bar to beat, and the leakage detector. Fails loudly if features leak the target.
`$PY ml/02_xgboost.py`	Trains the forecaster, writing `models/xgb_causal.json`.
`$PY ml/03_band_accuracy.py`	Does it make the right **call**, not just the right number. Confusion matrix per band.
`$PY ml/04_operating_point.py`	Fits the ACT NOW threshold to a stated recall target and reports what it costs in false alarms.
`$PY ml/04_operating_point.py --sweep`	The full cutoff curve, 30 to 90, so you can pick your own.
`$PY ml/04_operating_point.py --cutoff 70`	Measures one specific cutoff. Defaults to whatever `api/model.py` ships.
`$PY ml/06_intervals.py`	Empirical prediction intervals from observed error, written to `reports/intervals.json`.
`$PY ml/07_sequence.py`	Tests whether a sequence model beats tabular XGBoost here. It does not, by 0.30 m.

6. Verification and audits

COMMAND	WHAT IT DOES
`$PY api/test_smoke.py`	90 API assertions, no pytest, no network. Uses a throwaway database.
`$PY tools/audit_web.py`	Eight frontend checks, including whether a `hidden` attribute actually hides.
`$PY api/verify_model.py`	Replays held-out rows through the **live serving code** and compares to the published MAE. If it prints `DIVERGED`, do not quote the numbers.
`$PY api/verify_features.py`	Proves the serving feature path reproduces the training feature path exactly.
`$PY tools/validate.py`	Every data-quality check that could embarrass you in front of a judge.
`$PY tools/validate.py --no-plots`	Same, without writing figures to `reports/`.

COMMAND	WHAT IT DOES
`$PY tools/stamp_assets.py`	Re-hashes every CSS/JS URL in `web/`. **Run after editing anything in `web/`.**
`$PY tools/data_card.py`	Regenerates `DATA_CARD.md` from the database, so its numbers cannot drift from the data.

7. Release and export

COMMAND	WHAT IT DOES
`$PY tools/export_bootstrap.py`	Writes the Parquet files a fresh clone rebuilds the database from. Commit these.
`$PY tools/bootstrap.py`	Builds `data/jalaakar.db` from those Parquet files. No network, ~30 s.
`$PY tools/bootstrap.py --force`	Deletes and rebuilds. Refuses if `data/bootstrap/` is missing, rather than destroying the only copy.
`$PY tools/export_parquet.py`	Exports pipeline tables to `data/parquet/` for model training.
`$PY tools/freeze.py --mae 0.42`	Snapshots the database as `FROZEN_<timestamp>.db` with the accuracy it produced.
`$PY tools/make_fixtures.py`	Fabricates plausible data so one track can be developed without the other's output.
`$PY web/tools/gen_cracks.py`	Regenerates `web/assets/cracked-earth.svg`. Needs `scipy`; only run if you change that artwork.

8. Git and LFS

COMMAND	WHAT IT DOES
`git lfs install && git lfs pull`	Fetches the real Parquet files if a clone came down with 130-byte pointers.
`git lfs ls-files`	Confirms which files are actually stored in LFS.
`cd /tmp && git clone <url> jal-check && cd jal-check && make setup && make test`	Proves the repo works for a stranger. Expect `built — 490 MB` and `90/90 passed`.

9. The API — 26 endpoints

Interactive and always current at **http://localhost:8000/docs**.

Public

ENDPOINT	WHAT IT DOES
`GET /api/health`	Row counts per table, and whether the pipeline database opened.

ENDPOINT	WHAT IT DOES
`GET /api/figures`	The landing page's numbers with their provenance.
`GET /api/districts`	Districts that have wells behind them.
`GET /api/talukas?district=Nashik`	Talukas in a district, with well counts.
`GET /api/places?q=dind`	Typeahead over villages, talukas and cities.
`GET /api/places/resolve?place=Baglan`	Turns free text into an entity to score.
`GET /api/score?entity_type=taluka&entity_id=Baglan`	Scores one place. Add `&on=` for a date, `&lang=` for language.
`GET /api/score/{user_id}`	Scores that user's own registered place.
`GET /api/timeline?entity_id=MUM_ALL`	Score history — urban daily, rural one point per real CGWB reading.
`POST /api/signup`	Creates an account and returns their first score card.
`POST /api/reports`	Submits a Jal Mitra borewell reading; validates it against that well's history.
`GET /api/reports`	Community report log with its verdicts.
`POST /api/whatsapp/webhook`	Twilio inbound. Returns TwiML.
`POST /api/whatsapp/simulate`	The same conversation flow with no Twilio.
`POST /api/alerts/render`	The exact message for a place, in all three languages. Writes nothing.
`GET /api/alerts/templates`	Every alert string, for the native-reader review.
`GET /api/alerts/log`	Delivery log — what was really sent versus only rendered.

Signed in

ENDPOINT	WHAT IT DOES
`POST /api/auth/login`	Phone plus password. Returns a token valid 12 hours.
`GET /api/auth/me`	Who you are, whether you may send, and your own score.
`POST /api/auth/logout`	Revokes the token.

Senders only — government officials and verified society managers

ENDPOINT	WHAT IT DOES
`GET /api/admin/overview`	Every taluka and reservoir, scored, worst first, with subscriber reach. Add `?bucket=act_now`, `?bucket=monitor`, or `?refresh=true`.
`GET /api/admin/preview`	The wording — both bands, both audience types, three languages.
`POST /api/admin/broadcast`	One click: caution above 70, notice 41–70, each recipient scored for their own place. `{"dry_run": true}` writes nothing.

ENDPOINT	WHAT IT DOES
`POST /api/alerts/broadcast`	The older per-entity broadcast the demo page used.
`POST /api/alerts/send-demo`	Sends one alert for one place to one number.
`POST /api/alerts/preview`	Renders the alert for one existing user.
`POST /api/alerts/send`	Sends to one existing user by `user_id`.

10. The demo path, in order

```
make setup                          # once
make demo-user                      # 5 accounts, password jalaakar-demo
make run
```

1. **http://localhost:8000** — the landing page.
2. **/demo.html** — Nashik → Baglan (78, act now) against Nashik → Dindori (31, safe). Then the Urban tab: Mumbai on 29 Jun 2026 (90) versus today (0). This page looks the same to everyone.
3. **/login.html** — sign in as `9800000001` / `jalaakar-demo`. Officials land in the control room automatically.
4. **/admin.html** — the whole state worst-first. **Preview messages** before **Send now**; the preview writes nothing.
5. **/docs** — every endpoint, interactive.

Four things that bite

- **Run `tools/stamp_assets.py` after editing anything in `web/`.** A cached stylesheet against fresh HTML has broken this demo twice.
- **A SAFE score sends nothing.** Testing with Mumbai today looks broken and is not — use Baglan, or pass `--force`.
- **The Twilio sandbox expires after 72 hours of inactivity**, and every recipient must send the join phrase from their own phone first.
- **The rural record ends 2023-08-15.** Ask for a 2026 rural score and the API refuses with a reason rather than extrapolating. The urban track is current.

11. Every flag, by script

The sections above show the flags worth using. This is the complete list, generated from each script's own `argparse` definition, so nothing is left out. Defaults are shown where one exists; add `--help` to any script for its own wording.

SCRIPT	EVERY FLAG IT ACCEPTS
`tools/bootstrap.py`	`--force`
`tools/data_card.py`	`--out` `--mae`
`tools/delete_user.py`	`--phone` `--user-id` `--auto` `--purge-alerts` `--dry-run` `--yes` `--no-backup`
`tools/export_bootstrap.py`	`--no-weather`
`tools/export_parquet.py`	`--db` `--out`

SCRIPT	EVERY FLAG IT ACCEPTS
tools/freeze.py	`--mae` `--force`
tools/list_users.py	`--role` `--lang` `--place` `--since` `--auto-only` `--oldest` `--full` `--csv`
tools/reset_app_db.py	`--users` `--all` `--dry-run` `--yes` `--no-backup`
tools/send_test_alert.py	`--phone` `--place=Baglan` `--lang=mr` `--role=farmer` `--on=2023-05-15` `--force` `--dry-run`
tools/set_password.py	`--phone` `--password`
tools/validate.py	`--well` `--no-plots`
tools/verify_user.py	`--phone` `--list` `--revoke`
ingest/03_openmeteo.py	`--source=stub` `--csv=data/interim/mh_wells.csv` `--end` `--limit` `--dry-run` `--offline` `--no-load` `--no-soil`
ingest/03b_nasapower.py	`--csv=data/interim/mh_wells.csv` `--precision` `--start` `--end` `--dry-run` `--only-missing` `--no-load`
ingest/04_reservoirs.py	`--live` `--seed` `--interpolate` `--all` `--date`
ingest/05_interpolate.py	`--taluka` `--limit` `--validate` `--plot` `--no-load` `--from`
ingest/06_features.py	`--rural-only` `--urban-only` `--keep-warmup`
ingest/06b_features_causal.py	`--horizons=7,15,30` `--min-obs=20` `--require-weather`
ingest/07_stress.py	`--explain` `--date` `--calibrate`
ml/01_baseline.py	`--horizons=7,15,30` `--include-interpolated` `--out=reports/baseline_metrics.json`
ml/02_xgboost.py	`--target=delta_clim` `--per-horizon` `--weather-only` `--sample` `--dry-run` `--out=reports/xgboost_metrics.json` `--model-dir=models`
ml/03_band_accuracy.py	`--model=models/xgb_causal.json` `--target=delta_clim` `--out=reports/band_accuracy.json`
ml/04_operating_point.py	`--model=models/xgb_causal.json` `--target-recall=0.8` `--sweep` `--cutoff` `--out=reports/operating_point.json`
ml/07_sequence.py	`--lags=4` `--out=reports/sequence_metrics.json`
api/verify_features.py	`--n=400` `--split`
api/verify_model.py	`--n` `--metrics=reports/xgboost_metrics.json`

Two conventions that hold throughout: `--dry-run` **never writes**, and anything destructive (`delete_user.py`, `reset_app_db.py`) takes a backup first unless you pass `--no-backup`.